Card Pack JSON Format

Reference for authoring card packs for "No Filter" (a Cards Against Humanity–style party game)

This describes the on-disk / uploadable pack schema — the same format used for a Core Set upload, a Custom Set, and the bundled sample deck. Any file matching this schema can be uploaded through the app's admin Card Management page.

1. Top-level structure

A pack file is a single JSON array. Each element is one **pack** — a named bundle of white and black cards, roughly equivalent to one physical expansion.

```
[
  { "name": "...", "category": "...", "white": [ ... ], "black": [ ... ] },
  { "name": "...", "category": "...", "white": [ ... ], "black": [ ... ] }
]
```

There is no limit on the number of packs in one file, and no pack ever needs an explicit id — ids are assigned automatically by the app when the file is loaded.

2. Pack object

FIELD	TYPE	REQUIRED	NOTES
`name`	string	**Yes**	Display name of the pack, e.g. `"Office Life"`. Trimmed and capped at 120 characters. A pack with no name (empty after trimming) is silently dropped on upload.
`category`	string	Recommended	The **section** this pack is grouped under in the pack browser and the game-setup lobby — see §3 below. Freeform text, e.g. `"Homebrew"`, `"Pop Culture"`, `"NSFW"`. Any packs sharing the same `category` string are grouped together and can be filtered as one section.
`white`	array	-	This pack's white (answer) cards — see §4. May be omitted or empty for a black-cards-only pack.
`black`	array	-	This pack's black (prompt) cards — see §5. May be omitted or empty for a white-cards-only pack.
`official`	boolean	Legacy only	Only consulted as a fallback *when* `category` *is absent*: `true` resolves the section to `"Official"`, otherwise

			`"Uncategorized"`. New packs should just set `category` directly instead of relying on this.
`sheetName, mechanic`	string / number[]	Ignored	Both accepted for compatibility with an older schema but not read by anything in the app today. Safe to omit entirely in new packs.

A pack with both `white` and `black` empty (or absent) is dropped entirely on upload.

3. The `category` field — sections, in depth

"Section" in the app's UI **is** the `category` field. It is not a fixed list — it is whatever string each pack carries. The pack browser and the Lobby's pack picker group packs by this string, let the host narrow the list to one section at a time, and always show at least one section per file. There is no separate list of allowed section names to register anywhere; a new section is created just by giving a pack a `category` nobody has used yet.

> **Practical guidance for a new pack file:** pick a short, human-readable section name per theme or source — e.g. `"Base Game"`, `"Movies & TV"`, `"Internet Culture"`, `"Adult"`, `"Homebrew"` — and reuse the exact same string (case and spelling) across every pack that belongs in that section. Inconsistent capitalization or spelling (`"Homebrew"` vs `"homebrew"`) will create two separate sections instead of one.

If `category` is left out of a pack entirely, the app falls back to `"Official"` (if `official: true` was set) or `"Uncategorized"` — both are legacy fallbacks, not something to rely on for a new file. Always set `category` explicitly.

4. White cards

```
{ "text": "A haunted vending machine" }
```

FIELD	TYPE	REQUIRED	NOTES
`text`	string	Yes	The card's text. Trimmed of surrounding whitespace, capped at 300 characters (longer text is silently truncated). Empty text is dropped.

White cards need nothing else — no id, no pack reference. (Some older files include a per-card `pack` field; it's harmless to include but ignored — the card's pack is simply whichever pack object it's nested inside.)

5. Black cards

```
{ "text": "A totally reasonable request: please pass the _." }
{ "text": "The secret to a happy life: _ and _." }
{ "text": "Why did the chicken cross the road?" }
```

FIELD	TYPE	REQUIRED	NOTES
`text`	string	**Yes**	Same trim/300-char/empty-drop rules as white card text.
`pick`	number	Usually omit	How many white cards a player must submit for this prompt. See below — in almost every case, leave this out and let it be derived automatically from the blanks in the text.

Blanks and `pick` auto-correction

A blank in a black card's text is written as a single underscore character: `_`. The app derives `pick` from the actual number of underscores in the text, **overriding whatever `pick` value was supplied** whenever the text contains at least one blank. This exists specifically to self-heal mislabeled data (a real pick-3 card that shipped mislabeled pick-1 in a third-party file, for example) — so:

- A card with one `_` is always pick-1, two `_` is pick-2, three is pick-3, regardless of any `pick` field you provide.
- A card with **zero** blanks (an "open question" prompt like *"Why did the chicken cross the road?"*) has no blank to check against, so a supplied `pick` is trusted — clamped between 1 and 4, defaulting to 1 if omitted.
- The practical upshot: for a normal fill-in-the-blank prompt, just write the underscores correctly and omit `pick` entirely. Only set `pick` explicitly on a zero-blank open-question card that needs more than one answer.

`pick` is capped at **4** — the smallest hand size the app allows a room to be configured with — since asking for more cards than the smallest possible hand can hold would make a round unsubmittable.

6. Complete worked example

```
[
  {
    "name": "Office Life",
    "category": "Work & Money",
    "white": [
      { "text": "A calendar invite with no agenda" },
      { "text": "Reply-all" },
      { "text": "The Wi-Fi password taped to the printer" }
    ],
    "black": [
```

```
      { "text": "This meeting could have been _." },
      { "text": "HR would like to speak with you about _ and _." },
      { "text": "What's the real reason nobody wants Mondays?", "pick": 1 }
    ]
  },
  {
    "name": "Cursed Trivia",
    "category": "Homebrew",
    "white": [
      { "text": "A goose with a grudge" },
      { "text": "An extremely aggressive game of Uno" }
    ],
    "black": [
      { "text": "Grandma's famous recipe calls for _, _, and _." }
    ]
  }
]
```

Note in the example: the first pack's third black card sets `pick: 1` explicitly because it has zero blanks; every other black card just relies on its underscore count and needs no `pick` field at all.

7. Content notes

- **Duplicates across packs are fine at the file level.** A room-level "Exclude duplicate cards" toggle dedupes by trimmed/lowercased text across whichever packs a host selects — that happens at game time, not at upload time, so don't worry about de-duplicating a submitted file by hand.
- **A vulgar-language filter exists separately** (room settings: exclude vulgar cards, or vulgar-only) and runs against card text at game time. It has no bearing on how a pack file should be written — just write normal card text.
- **File size:** uploads are accepted up to 32MB, well beyond what a handful of new packs will need.
- **Two upload modes** exist in the admin UI: *add* (append these packs to whatever's already loaded) and *replace* (wipe and replace the entire card library with this file). Worth stating explicitly which mode a new file is meant for when handing it off.

> **Minimal valid pack, for reference:**
>
> ```
> [
>   {
>     "name": "Test Pack",
>     "category": "Test",
>     "white": [{ "text": "A test card" }],
>     "black": [{ "text": "Fill in the _." }]
> ```

```
  }
]
```
